LLM Threat Activation Dataset (Meta-Llama-3-8B)

Dataset Summary

这是一个专为**大型语言模型解码层威胁检测**任务设计的多维激活数据集。该数据集通过向 Meta-Llama-3-8B 模型注入多种攻击向量，捕获了每个解码器层的隐藏状态（hidden states），用于训练和评估能够识别 LLM 内部异常行为的分类器。

属性	值
模型	LLM-Research/Meta-Llama-3-8B
指令数量	10,000 条
解码层数	32 层
威胁类别	5 类
总样本数	1,600,000 条
数据格式	Apache Parquet
数据集大小	~20 GB
输入数据源	AI-ModelScope/alpaca-gpt4-data-en (Alpaca GPT-4)

Dataset Schema

数据集采用 Apache Parquet 格式存储，包含以下字段：

字段名	类型	描述
`sample_id`	string	唯一样本标识符，格式: `{instruction_id:05d}_L{layer:03d}_T{label}`
`model_name`	string	模型短名称 (e.g., "Meta-Llama-3-8B")
`instruction_id`	int32	指令索引 (0 ~ 9999)
`layer_index`	int32	解码器层索引 (0 ~ 31)
`label`	int32	威胁类别标签 (0 ~ 4)
`label_name`	string	威胁类别名称
`hidden_state_vector`	list	最后一词元的隐藏状态向量，维度=4096

Threat Categories (标签定义)

数据集包含 5 种威胁类别，用于模拟不同类型的攻击或故障场景：

标签	名称	描述	攻击类型
0	`honest`	正常的解码器层输出，作为基准样本	正常行为
1	`silent_precision_downgrade`	模拟 INT4 对称量化往返（静默精度降级攻击）	量化攻击
2	`identity_forgery`	返回层输入而非层输出（跳跃层/身份伪造攻击）	层替换攻击
3	`random_noise`	高斯随机噪声，均值/标准差匹配真实输出（拜占庭故障）	随机故障
4	`adversarial_perturbation`	真实输出叠加 5% 标准差的高斯扰动（对抗性攻击）	微小扰动攻击

攻击向量详解

1. Honest (label=0)

正常的模型前向传播输出，作为分类任务的负样本（正常类）。

2. Silent Precision Downgrade (label=1)

模拟低比特量化攻击：

- 计算绝对最大值 `abs_max = max(|output|)`
- 量化尺度 `scale = abs_max / 7.0`
- 量化: `quantized = round(output / scale)` 并裁剪到 [-8, 7]
- 反量化: `output' = quantized * scale`

3. Identity Forgery (label=2)

模拟跳跃层攻击，直接将层的输入作为输出返回，绕过该层的计算。

4. Random Noise (label=3)

模拟拜占庭故障或完全损坏的层：

- 计算真实输出的均值 `μ` 和标准差 `σ`
- 生成高斯噪声: `noise ~ N(μ, σ)`
- 完全替换原始输出

5. Adversarial Perturbation (label=4)

模拟微小对抗性扰动：

- 计算真实输出的标准差 `σ`
- 生成微小扰动: `perturbation ~ N(0, 0.05σ)`
- 叠加扰动: `output' = output + perturbation`

Data Generation Process

生成流程

1. **模型加载**: 从 ModelScope 加载 Meta-Llama-3-8B (BF16 精度)
2. **数据加载**: 从 Alpaca GPT-4 数据集加载 10,000 条指令
3. **钩子注册**: 在每个解码器层注册 forward hook，捕获输入/输出隐藏状态
4. **逐指令处理**:
    - 对每条指令进行 tokenization (最大长度 2048)
    - 执行前向传播（无梯度）
    - 捕获每层最后一个词元的输入和输出隐藏状态
    - 为每层生成 5 种威胁变体
    - 将结果写入 Parquet 文件
5. **断点续传**: 支持通过 `resume_state.json` 从中断处恢复

捕获机制

- **捕获位置**: 每个 Transformer 解码器层的输入和输出
- **捕获方式**: PyTorch forward hook
- **向量选择**: 每个序列的最后一个词元 (`[:, -1, :]`)
- **数据类型**: `float32`（从 BF16 转换）
- **数值检查**: 自动过滤包含 NaN/Inf 的样本

输出文件结构

```
output/
├── part_000000.parquet    # 第 0 批数据 (~10,000 条记录)
├── part_000001.parquet    # 第 1 批数据
├── ...
├── part_00XXXX.parquet    # 最后一批数据
└── resume_state.json      # 断点续传状态文件
```

Dataset Statistics

基本信息

```
总指令数: 10,000
模型层数: 32
每指令每层样本数: 5 (5种威胁类别)
总样本数: 10,000 × 32 × 5 = 1,600,000（精确值）
隐藏维度: 4096
向量数据类型: float32
单条记录大小: ~16 KB (向量) + 元数据
```

类别分布

数据集采用平衡采样策略，每个类别的样本数大致相等：

类别	样本数	占比
honest	320,000	20%
silent_precision_downgrade	320,000	20%
identity_forgery	320,000	20%
random_noise	320,000	20%
adversarial_perturbation	320,000	20%

层分布

每个解码器层 (0-31) 的样本数均匀分布。

Usage

加载数据

使用 PyArrow

```
import pyarrow.parquet as pq
import pandas as pd

# 加载单个 part 文件
table = pq.read_table("output/part_000000.parquet")
df = table.to_pandas()

# 或使用 pandas 直接读取
df = pd.read_parquet("output/part_000000.parquet")
```

加载整个数据集

```
import pyarrow.dataset as ds

# 加载整个目录
dataset = ds.dataset("output", format="parquet")
table = dataset.to_table()
df = table.to_pandas()
```

使用 HuggingFace Datasets

```
from datasets import load_dataset
```

```
# 假设数据已上传到 HuggingFace Hub
dataset = load_dataset("your-username/llama3-threat-activation",
split="train")
```

标准使用模式

1. 威胁检测分类器训练

```
import torch
from torch.utils.data import Dataset, DataLoader

class ThreatActivationDataset(Dataset):
    def __init__(self, parquet_path):
        self.df = pd.read_parquet(parquet_path)

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        vector = torch.tensor(row['hidden_state_vector'],
dtype=torch.float32)
        label = torch.tensor(row['label'], dtype=torch.long)
        return vector, label

# 创建 DataLoader
dataset = ThreatActivationDataset("output/")
dataloader = DataLoader(dataset, batch_size=256, shuffle=True)

# 训练分类器（示例：简单的 MLP）
class ThreatClassifier(torch.nn.Module):
    def __init__(self, input_dim=4096, num_classes=5):
        super().__init__()
        self.classifier = torch.nn.Sequential(
            torch.nn.Linear(input_dim, 1024),
            torch.nn.ReLU(),
            torch.nn.Dropout(0.3),
            torch.nn.Linear(1024, 256),
            torch.nn.ReLU(),
            torch.nn.Linear(256, num_classes)
        )

    def forward(self, x):
        return self.classifier(x)
```

2. 按层分析

```
# 分析特定层的威胁检测性能
layer_0_data = df[df['layer_index'] == 0]
layer_last_data = df[df['layer_index'] == 31]

# 统计各层各类别分布
layer_label_dist = df.groupby(['layer_index',
'label_name']).size().unstack()
```

3. 数据划分建议

```
from sklearn.model_selection import train_test_split

# 按 instruction_id 划分，确保同一指令的样本不会同时出现在训练集和测试集
train_ids, test_ids = train_test_split(
    df['instruction_id'].unique(),
    test_size=0.2,
    random_state=42
)

train_df = df[df['instruction_id'].isin(train_ids)]
test_df = df[df['instruction_id'].isin(test_ids)]
```

Citation

如果您在研究中使用此数据集，请引用：

```
@dataset{llama3_threat_activation_2025,
  title = {LLM Threat Activation Dataset: Meta-Llama-3-8B},
  author = {Your Name},
  year = {2025},
  publisher = {ModelScope/HuggingFace},
  url = {https://modelscope.cn/datasets/...}
}
```

Limitations

1. **模型特定性**: 本数据集基于 Meta-Llama-3-8B 生成，向量维度和层数特定于该模型架构
2. **指令分布**: 基于 Alpaca GPT-4 数据集的指令分布，可能不涵盖所有类型的用户查询
3. **攻击模拟**: 威胁类别为模拟攻击，可能与真实攻击有差异
4. **单模型**: 仅包含单个模型的数据，跨模型泛化需额外验证

License

本数据集的生成代码采用相应开源许可证。原始模型 (Meta-Llama-3-8B) 受其自身许可证约束。原始 Alpaca 数据集遵循其原始许可证。

Contact

如有问题或建议，请通过以下方式联系：

- GitHub Issues: [项目仓库]
- Email: [联系邮箱]

Acknowledgments

- 基础模型: Meta-Llama-3-8B by Meta AI
- 指令数据集: Alpaca GPT-4 基于 Stanford Alpaca
- 生成工具: threat_dataset 自定义数据集生成框架